Appium

Mobile web automation

Instalação, WebdriverIO, gestos, contexts, POM e testflow-appium

Android · iOS · TypeScript · Allure

O que é o Appium?

Framework open-source de automação mobile que usa o protocolo **WebDriver** — funciona com apps nativos, híbridos e **mobile web** (Chrome/Safari).

- **Appium 2** — arquitetura modular com drivers plugáveis
- Drivers: `uiautomator2` (Android) · `xcuitest` (iOS)
- Cliente: **WebdriverIO 9** + Mocha neste projeto
- TestFlow via browser mobile — mesma app dos suites Cypress/Playwright
- Screenshots on failure, Allure report, tags Mocha (`@smoke`, `@appium`)

Por que Appium + WebdriverIO?

Dispositivos reais

Emulador Android, Simulator iOS ou device físico — gestos touch nativos.

Mobile web

Chrome no Android, Safari no iOS — POM com `data-testid`.

Appium 2 local

`@wdio/appium-service` sobe o server automaticamente no `wdio run`.

Allure + CI

Relatório HTML no GitHub Pages; smoke Android no PR gate.

Pré-requisitos

Ambiente

- **Node.js** 20+
- **Java 17+** (Allure report)
- **Android:** SDK, emulator, Chrome
- **iOS:** macOS, Xcode, Simulator
- TestFlow na porta `5050`

```
$ node --version
v20.x.x

$ docker run -p 5050:5050 qaschool/testflow:latest
# npm run check:testflow
```

Instalação do ambiente por SO

Antes de clonar: **Node 20+**, **Git**, **Docker** (TestFlow) e stack mobile conforme plataforma alvo.

macOS — Android + iOS

Base

```
brew install node@20 git
brew link --overwrite node@20
brew install --cask docker android-studio

# Java 17 (Allure)
brew install openjdk@17
```

Appium drivers (no projeto)

```
npm install
npm run appium:drivers   # uiautomator2 + xcuitest
npm run appium:doctor    # valida drivers

# iOS — Xcode + Simulator
xcode-select --install
open -a Simulator
```

Windows — Android

```
winget install OpenJS.NodeJS.LTS
winget install Git.Git
winget install Docker.DockerDesktop

# Android Studio + SDK Manager
# ANDROID_HOME → %LOCALAPPDATA%\Android\Sdk
```

```
npm install
npm run appium:drivers

# Emulador: Pixel 7, API 34
# adb reverse tcp:5050 tcp:5050  # CI pattern
```

iOS exige macOS — use CI ou Mac para `npm run test:ios`.

Linux — Android (CI)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs git openjdk-17-jdk

npm install
npm run appium:drivers
```

```
# GitHub Actions usa reactivecircus/android-emulator-runner
# KVM + API 34 + pixel_7 profile
# scripts/ci-android-smoke.sh
```

Instalação — testflow-appium

1 Clone e instale

```
git clone …/testflow-appium && cd testflow-appium && npm install
```

2 Drivers Appium

```
npm run appium:drivers
```

3 Configure env

```
cp .env.example .env
```

4 Suba TestFlow e rode

```
docker run -p 5050:5050 qaschool/testflow:latest
```

`npm run test:api` ou `npm run test:smoke`

URLs mobile — loopback

Emuladores não resolvem `localhost` como o desktop.

Plataforma	Base URL padrão
Android emulator	`http://10.0.2.2:5050`
iOS simulator	`http://127.0.0.1:5050`
CI (adb reverse)	`http://127.0.0.1:5050`
API only / desktop	`http://localhost:5050`

Override: `BASE_URL`, `ANDROID_BASE_URL`, `IOS_BASE_URL`, `API_BASE_URL`

Estrutura do projeto

```
testflow-appium/
├── wdio.conf.ts          # Android
├── wdio.ios.conf.ts      # iOS Safari
├── wdio.api.conf.ts      # REST-only
├── wdio.shared.conf.ts   # Appium service + Allure
├── pages/                # Page Object Model
├── support/
│   ├── auth.ts           # login API/UI + sessionStorage
│   ├── selectors.ts      # data-testid helpers
│   ├── navigation.ts     # openAppPath
│   └── helpers/          # gestures, waits, visual
├── tests/                # specs por feature
└── scripts/              # CI + Allure
```

- `pages/` — ações e assertions por tela
- `support/auth.ts` — seed de sessão via API
- `tests/*.spec.ts` — Mocha + expect-webdriverio
- Tags: `@smoke` · `@regression` · `@appium` · `@api`

Configuração — wdio.conf.ts

```ts
const androidCaps = {
  platformName: 'Android',
  browserName: 'Chrome',
  'appium:automationName': 'UiAutomator2',
  'appium:deviceName': process.env.ANDROID_DEVICE ?? 'Android Emulator',
  'appium:chromedriverAutodownload': true,
  'appium:pageLoadStrategy': 'eager',
  'wdio:enforceWebDriverClassic': true,
}

export const config = {
  ...sharedConfig,
  capabilities: [androidCaps],
}
```

`sharedConfig`: Appium service, Mocha, Allure reporter, screenshot on failure.

Seu primeiro teste

```ts
// tests/auth/login.spec.ts
import { LoginPage } from '../../pages/LoginPage'
import { DEMO_EMAIL, DEMO_PASSWORD } from '../../support/config'

describe('Authentication @regression', () => {
  const login = () => new LoginPage()

  beforeEach(async () => {
    await login().visit()
  })

  it('logs in via UI and redirects to dashboard', async () => {
    await login().loginWith(DEMO_EMAIL, DEMO_PASSWORD)
    await login().shouldRedirectToDashboard()
  })
})
```

Navegação & browser

API	Uso no TestFlow
`browser.url(url)`	Abrir login, dashboard via `openAppPath`
`browser.getUrl()`	Assert redirect pós-login
`browser.getTitle()`	Smoke — título da página
`browser.execute(fn)`	sessionStorage, DOM queries, userAgent
`browser.waitUntil(fn)`	document.readyState, poll custom
`browser.takeScreenshot()`	Allure attachment on failure
`browser.saveScreenshot(path)`	Visual baselines, device spec
`browser.capabilities`	platformName, device metadata

Elementos — find & interact

API	Exemplo TestFlow
`browser.$(selector)`	`[data-testid="login-email"]`
`browser.$$(prefix)`	Múltiplos elementos por prefixo
`.click()`	Submit, sidebar, nav links
`.setValue(text)`	Email, senha, busca
`.clearValue()`	Limpar campo antes de fill
`.getText()`	KPIs, erros, labels
`.getAttribute(name)`	Classe active na nav
`.waitForDisplayed()`	Explicit wait por elemento
`.waitForEnabled()`	Campo habilitado antes de interagir
`.isDisplayed()` / `.isExisting()`	Condicionais em locators spec

Assertions — expect-webdriverio

Assertion	Exemplo
`toBeDisplayed()`	Page root, KPI card, modal
`toBeEnabled()`	Campo password após wait
`toHaveUrl(...)`	Redirect pós-login / logout
`toHaveText(...)`	Mensagem de erro no login
`expect(browser).toHaveUrl(...)`	URL contém `/web/team.html`
`assert.strictEqual`	Contratos API (Mocha + Axios)

Helpers — support/selectors.ts

Seletores data-testid	
`byTestId(id)`	CSS `[data-testid="…"]`
`$(testId)`	Atalho para `browser.$`
`waitForTestId(id)`	waitForExist + waitForDisplayed
`clickTestId(id)`	Click com wait implícito
`fillTestId(id, value)`	click → clear → setValue
`getTextTestId(id)`	Texto de elemento estável
support/helpers/waits.ts	
`waitForUrl(regex)`	Poll até URL match
`waitForText(testId, text)`	Poll até texto aparecer
`pollUntil(fn, pred)`	Retry genérico com intervalo

Gestos touch — performActions

```ts
// support/helpers/gestures.ts
export async function swipeUp({ percent = 0.4, speed = 600 } = {}) {
  const { width, height } = await browser.getWindowSize()
  await browser.performActions([{
    type: 'pointer', id: 'finger1',
    parameters: { pointerType: 'touch' },
    actions: [
      { type: 'pointerMove', duration: 0, x: width/2, y: height*0.75 },
      { type: 'pointerDown', button: 0 },
      { type: 'pointerMove', duration: speed, x: width/2, y: height*(1-percent) },
      { type: 'pointerUp', button: 0 },
    ],
  }])
  await browser.releaseActions()
}
```

`swipeUp` `swipeDown` `longPress` `scrollIntoView`

Device capabilities

Função / API	Spec
`hideKeyboardIfShown()`	`device.spec.ts` — teclado software
`setOrientation('PORTRAIT'|'LANDSCAPE')`	Rotação portrait/landscape
`browser.isKeyboardShown()`	Detectar teclado aberto
`browser.hideKeyboard()`	Fechar teclado Android
`browser.setOrientation(...)`	Wrapper Appium
`browser.saveScreenshot(path)`	Screenshot sob demanda
`mobile: backgroundApp`	Simula background (native; web ignora)

Context switching

```ts
// support/helpers/gestures.ts
export async function listContexts() {
  return (await browser.getContexts()).map(String)
}

export async function switchToWebContext() {
  const web = (await listContexts())
    .find((ctx) => ctx.includes('WEBVIEW') || ctx === 'WEBVIEW')
  if (web) await browser.switchContext(web)
}

// tests/contexts/webview.spec.ts
const testIds = await browser.execute(() =>
  Array.from(document.querySelectorAll('[data-testid]'))
    .slice(0, 5).map(el => el.getAttribute('data-testid'))
)
```

Estratégias de locator

Estratégia	Exemplo
CSS / data-testid	`browser.$(byTestId('login-email'))`
XPath	`//input[@data-testid="login-email"]`
Atributo composto	`button[data-testid="login-submit"]`
Explicit wait	`.waitForDisplayed({ timeout: 10000 })`

Preferência: `data-testid` — mesma convenção Cypress/Playwright.

Auth helpers

```
// support/auth.ts
export async function loginViaApi() {
  const session = await fetchAuthToken()  // Axios POST /api/auth/login
  await seedSessionOnOrigin(() => injectAuthSession(session))
  await openAppPath('/web/dashboard.html')
  await waitForTestId('page-dashboard')
}

export async function visitWithToken(path, token) {
  await seedSessionOnOrigin(() => injectAuthSession({ token, ... }))
  await openAppPath(path)
}
```

`injectAuthSession` grava `sandbox-auth` e `sandbox-token` no sessionStorage.

API testing — sem device

```
// tests/api/auth.api.spec.ts — Mocha + Axios
describe('API — health & auth @api @smoke', () => {
  it('GET /health returns 200', async () => {
    const res = await apiClient().get('/health')
    assert.strictEqual(res.status, HTTP.OK)
  })

  it('POST /api/auth/login returns token', async () => {
    const res = await apiClient().post('/api/auth/login', {
      email: DEMO_EMAIL, password: DEMO_PASSWORD,
    })
    validateSchema(res.data, { token: 'string', user: 'object' })
  })
})
```

```
npm run test:api   # não precisa de emulador
```

Visual baselines

```
// support/helpers/visual.ts
export async function assertScreenshotMatches(name, maxMismatchPercent = 5) {
  const baselinePath = `test-results/visual-baselines/${name}.png`
  if (!fs.existsSync(baselinePath)) {
    await saveBaseline(name)  // cria baseline na 1ª execução
    return
  }
  await browser.saveScreenshot(`${name}-current.png`)
  // compara tamanho de arquivo como proxy de diff
}
```

`npm run test:visual` `@visual`

Page Object Model (POM)

Spec (.spec.ts)

- describe / it (Mocha)
- Given / When / Then
- Sem seletores espalhados

→

Page Object

- `testId('login-email')`
- Actions (`loginWith`)
- Assertions (`shouldRedirect…`)

Page Objects do TestFlow

- `LoginPage.ts` auth, validação, API toggle
- `DashboardPage.ts` KPIs, activity, new run
- `TeamPage.ts` tabela, invite, inline edit
- `SettingsPage.ts` forms, toggles, save
- `WizardPage.ts` multi-step, review
- `ShellPage.ts` nav, theme, mobile shell

Cobertura da suíte

Suite	Spec	Tag
Smoke	navigation.spec.ts	`@smoke`
Auth	login.spec.ts	`@regression`
Dashboard / Team / Settings	*.spec.ts	`@regression`
Gestures	gestures.spec.ts	`@appium`
Device	device.spec.ts	`@appium`
Contexts	webview.spec.ts	`@appium`
Locators	strategies.spec.ts	`@appium`
Visual	visual.spec.ts	`@visual`
API	auth.api.spec.ts	`@api`

Todos os comandos npm

Comando	Descrição
Setup	
`npm run appium:drivers`	Instala uiautomator2 + xcuitest
`npm run appium:doctor`	Valida drivers Appium
`npm run check:testflow`	Health check porta 5050
Execução	
`npm test`	Suite Android completa
`npm run test:android`	Android mobile web
`npm run test:ios`	iOS Safari (macOS)
`npm run test:api`	REST — sem emulador
`npm run test:smoke`	Só smoke specs
`npm run test:gestures`	Gestos touch
`npm run test:device`	Teclado, orientação

Todos os comandos npm

Comando	Descrição
Execução	
`npm run test:contexts`	Context switching
`npm run test:locators`	Estratégias de locator
`npm run test:grep:smoke`	Filtra tag `@smoke`
`npm run test:grep:appium`	Filtra tag `@appium`
Relatório	
`npm run allure:generate`	Gera HTML Allure
`npm run allure:open`	Abre relatório
`npm run report`	generate + open
Slides	
`npm run slides`	Servidor local :3336
`npm run slides:pdf`	Exporta `appium-intro-slides.pdf`

Allure Report

```
npm run test:smoke
npm run allure:generate   # Java 17+
npm run allure:open

# atalho
npm run report
```

- Screenshots anexados em falha (`afterTest`)
- URL atual no attachment
- Env vars no report (platform, CI, URLs)
- GitHub Pages: push em `main` → `pages.yml`

CI — GitHub Actions

Job api

REST smoke contra TestFlow Docker — gate rápido

android-smoke

Emulator API 34, KVM, `ci-android-smoke.sh`

publish / deploy

Landing, guides, slides + Allure (or placeholder) na `main`

Artefatos

Screenshots + allure-results em falha

Boas práticas

- **Page Objects** — locators `data-testid` centralizados
- `loginViaApi` no `beforeEach` — specs focam na feature
- `waitForTestId` — explicit wait antes de interagir
- URLs por plataforma em `support/config.ts` — não hardcode localhost
- Gestos via `performActions` — W3C Actions API
- `api` e `android-smoke` em paralelo no CI
- Tags Mocha — filtrar `@smoke`, `@appium` no CI
- IDs rastreáveis `[TC-xxxx]` em `testCases.ts`
- Um cenário por `it()` — falhas legíveis no Allure

Próximos passos

Você já sabe instalar Appium, configurar WebdriverIO e rodar mobile web no TestFlow.

Slides (PDF)
appium-intro-slides.pdf

Guia completo (PT)
Instalação e referência

Documentação
appium.io

WebdriverIO
webdriver.io

TestFlow
App sob teste